CS4523 - Final Report

A Comparative Study of Deterministic and Randomized Approaches to Column Subset Selection for Low-Rank Matrix Approximation

Date of Submission:
21st September 2026

Group Name: Randomizers

Group Members:

A.A.D.H.S. Amarasinghe - 220023U
K.B.I. Sandiw - 220573E

Table of Content

1. Introduction..... 1
2. Objectives..... 2
3. Methodology..... 2
4. Results and Discussion..... 18
5. Conclusion..... 26
6. References..... 27

1. Introduction

Large-scale matrices arise in many areas of machine learning, data analysis, and scientific computing, where processing and storing the complete matrix can become computationally expensive. Low-rank approximation techniques address this problem by representing a matrix using a lower-dimensional structure. The **Column Subset Selection Problem** (CSSP) considers a more constrained form of approximation: given a matrix $A \in \mathbb{R}^{m \times n}$ and a target k, the objective is to select k actual columns of A that can represent the full matrix with low approximation error. Since finding an optimal subset can be computationally difficult, practical deterministic and randomized algorithms are used to obtain effective column subsets.

This project investigates how randomization affects the solution of CSSP at two stages of the computational pipeline. For basis construction, deterministic truncated Singular Value Decomposition (SVD) is compared with randomized SVD based on random sketching [3]. For column selection, **Osinsky's deterministic method** is compared with **Adaptive Randomized Pivoting** (ARP), introduced by Cortinovis and Kressner as a randomized approach closely related to the deterministic selection procedure [1]. This provides a direct setting for studying the consequences of replacing deterministic computations with randomized alternatives while solving the same underlying problem.

The study is further extended with **Rejection ARP**, a rejection-sampling implementation of ARP designed to improve the efficiency of randomized column selection [4]. Combining deterministic and randomized SVD with Osinsky, ARP, and Rejection ARP results in six CSSP pipelines. This design allows randomization in basis construction and column selection to be examined separately as well as in combination.

The project compares these approaches through analytical and experimental evaluation, with emphasis on **approximation quality**, **computational cost**, **scalability**, and the **variability** introduced by randomized methods. Both controlled synthetic matrices and real-world datasets are considered so that the algorithms can be studied under different matrix sizes and structures. The overall aim is to identify the practical trade-offs that arise when randomization is introduced into different stages of the CSSP pipeline.

2. Objectives

The main objective of this project is to investigate how introducing randomization at different stages of a Column Subset Selection Problem (CSSP) pipeline affects approximation quality, computational performance, and solution variability.

The specific objectives are to:

- Study the theoretical and algorithmic foundations of CSSP, low-rank approximation, deterministic and randomized SVD, Osinsky's column-selection method, Adaptive Randomized Pivoting (ARP), and Rejection ARP.
- Implement and validate **six CSSP pipelines** formed by combining deterministic and randomized SVD with Osinsky, ARP, and Rejection ARP column-selection methods.
- Experimentally evaluate the algorithms using controlled synthetic matrices and real-world datasets with different sizes and structural properties, including rank, noise, singular-value decay, and sparsity.
- Compare the deterministic and randomized approaches in terms of approximation error, execution time, scalability, and, where relevant, memory requirements.
- Investigate the effect of randomization through repeated trials with different random seeds and examine the trade-offs introduced by randomization in basis construction, column selection, and their combination.

3. Methodology

3.1 CSSP Pipeline Design and Implementation

3.1.1 Overall Pipeline

The implementation was organized as a modular CSSP pipeline so that different basis-construction and column-selection methods could be evaluated within the same computational framework. Given an input matrix $A \in \mathbb{R}^{m \times n}$ and a target rank k, the pipeline first constructs a k-dimensional right singular subspace represented by a basis matrix $V \in \mathbb{R}^{n \times k}$. A column-selection algorithm then uses A or V or both, depending on the method, to determine an index set J containing k distinct column indices. The corresponding columns of the original matrix form the selected matrix $C=A[:,J]$.

The overall computational flow is as follows:

$$A \rightarrow \textit{Basis Construction} \rightarrow V \rightarrow \textit{Column Selection} \rightarrow J \rightarrow C=A[:,J] \rightarrow \textit{Reconstruction} \rightarrow \widehat{A} \rightarrow \textit{Evaluation}$$

After column selection, the selected columns are used to construct an approximation $\widehat{A}$ of the original matrix. The quality of the selected subset can then be evaluated by comparing A and $\widehat{A}$ using the approximation metrics defined for the experiments. Separating basis construction, column selection, reconstruction, and evaluation into distinct stages allows alternative algorithms to be substituted at individual stages while keeping the remainder of the pipeline unchanged.

This modular design is particularly important for the comparisons in this project. Deterministic and randomized basis-construction methods can be paired with different column-selection algorithms without changing the reconstruction and evaluation procedure. Consequently, the effect of introducing randomization at the basis-construction stage, the column-selection stage, or both stages can be investigated within a common pipeline.

3.1.2 Basis Construction

The first stage of the CSSP pipeline constructs a k-dimensional basis $V \in \mathbb{R}^{n \times k}$ that approximates the dominant row space of the input matrix A. Two approaches were implemented: **deterministic truncated Singular Value Decomposition** (SVD) and **randomized SVD**. Both methods produce a basis with the same dimensions, allowing the subsequent column-selection algorithms to operate within a common pipeline.

1. Deterministic SVD

For the deterministic approach, the singular value decomposition of A is expressed as:

$$A = U\Sigma V^T$$

The singular values are ordered from largest to smallest, and the first k right singular vectors are retained. The resulting basis is:

$$V_k = V[:, 1{:}k],$$

where $V \in \mathbb{R}^{n \times k}$. This basis represents the dominant k-dimensional right singular subspace of A and is used as the deterministic basis for column selection.

2. Randomized SVD

The randomized alternative follows the randomized low-rank approximation approach described by *Halko, Martinsson, and Tropp* [3]. Instead of directly computing the complete singular value decomposition of the original matrix, a Gaussian random projection is used to construct a lower-dimensional subspace that captures its dominant structure.

A random test matrix Ω is generated and used to form a sample of the range of A. Oversampling is used to provide additional dimensions beyond the requested rank k, while power iterations can be applied to improve the approximation when the singular values decay slowly. An orthonormal basis for the sampled subspace is then constructed, after which an

SVD is performed on the resulting smaller projected matrix. The first k approximate right singular vectors form the randomized basis V_k.

The randomized implementation accepts a fixed random seed, allowing individual runs to be reproduced while also enabling repeated experiments with different seeds. Both deterministic and randomized basis construction therefore produce an $n \times k$ basis that can be passed to the same column-selection stage, making it possible to study the effect of basis randomization independently of the selection algorithm.

3.1.3 Column Selection

Once the basis $V \in \mathbb{R}^{n \times k}$ has been constructed, the next stage selects k indices J corresponding to columns of the original matrix A. Three selection methods were implemented: **Osinsky's deterministic method**, **Adaptive Randomized Pivoting**, and **Rejection ARP**. These methods provide one deterministic and two randomized approaches to the same selection stage.

1. Osinsky's Deterministic Method

Osinsky's method provides the deterministic column-selection approach used in this project. Starting from the matrix A and its right basis V_k, the algorithm constructs the subset sequentially. At each step, candidate columns are assessed using residual-based quantities involving the matrix and the current basis, and a column is selected deterministically according to the resulting criterion. The residual quantities are then updated before the next selection is made. This process continues until k distinct column indices have been selected.

The implementation also applies numerical scaling to the quantities involved in the selection criterion to reduce the risk of overflow or underflow when matrices contain values of very different magnitudes. Because no random sampling is involved, the same input matrix and basis produce the same selected subset.

2. Adaptive Randomized Pivoting

Adaptive Randomized Pivoting (ARP), introduced by *Cortinovis and Kressner* [1], replaces deterministic pivot selection with adaptive randomized sampling. The method can be viewed as a randomization of Osinsky's deterministic CSSP selection procedure.

Rather than selecting the next index through a deterministic criterion, ARP assigns sampling probabilities using the residual information contained in the current basis. An index is sampled according to these probabilities, after which a Householder transformation is applied to update the basis for the next selection step. The probabilities therefore change adaptively as indices are selected. This procedure is repeated until k distinct indices have been obtained.

Since ARP contains a random sampling step, different random seeds can result in different selected subsets. A controllable random seed was therefore included in the implementation to

support both reproducible individual runs and repeated trials for studying the variability introduced by randomized selection.

3. Rejection ARP

Rejection ARP is an alternative implementation of ARP based on the rejection-sampling approach described by *Epperly* [4]. It retains the randomized selection principle of ARP but changes how samples are generated in order to make the sampling procedure more computationally efficient.

Instead of repeatedly constructing and sampling directly from the evolving adaptive distribution, Rejection ARP uses fixed leverage-score-based proposal probabilities. Candidate indices are drawn from this proposal distribution and accepted or rejected according to probabilities determined by the current residual information. Accepted indices become pivots, while rejected proposals are discarded and sampling continues. The implementation processes proposals in blocks and records quantities such as the total number of proposals and the resulting acceptance rate.

The three selection methods therefore represent closely related alternatives within the same CSSP pipeline. Osinsky provides the deterministic baseline, ARP introduces adaptive randomized pivot selection, and Rejection ARP provides a rejection-sampling implementation of the randomized approach. Because each method ultimately produces an index set J of size k, the selected columns can subsequently be formed in the same way, allowing the reconstruction and evaluation stages to remain common across three methods.

3.1.4 Matrix Reconstruction

After the index set J has been selected, the corresponding columns of the original matrix are extracted to form $C=A[:,J]$, where $C \in \mathbb{R}^{m \times k}$. Two reconstruction approaches were implemented: orthogonal reconstruction and oblique reconstruction. Both produce an approximation $\widehat{A}$ of the original matrix but use the selected columns differently.

1. Orthogonal Reconstruction

The orthogonal reconstruction projects the columns of A onto the column space spanned by the selected matrix C. It is computed as:

$$\widehat{A}_{\text{orth}} = CC^{+}A$$

where C^{+} denotes the Moore-Penrose pseudoinverse of C. The matrix CC^{+} is the orthogonal projector onto the column space of C. Therefore, this reconstruction measures how well the subspace spanned by the selected columns can approximate the complete matrix.

Orthogonal reconstruction is used as the primary reconstruction method for the main approximation-quality experiments in this project. This provides a common evaluation procedure independent of which basis-construction or column-selection method produced the selected subset.

2. Oblique Reconstruction

An oblique reconstruction was also implemented using the selected rows of the basis V_k. For a selected index set *J*, it is given by:

$$\widehat{A}_{\text{oblique}} = A_{:,J}\left(V_k[J,:]\right)^{-T} V_k^T$$

Here, $V_k[J,:] \in \mathbb{R}^{k \times k}$ contains the rows of V_k corresponding to the selected column indices. The reconstruction therefore depends not only on the selected columns *A[:,J]*, but also on the basis used during column selection. This form is particularly relevant to the theoretical formulation of ARP [1].

Both reconstruction methods are calculated by the implementation, allowing their approximation errors to be examined when required. However, the orthogonal reconstruction is used as the standard measure of CSSP approximation quality in the main experiments, while the oblique reconstruction is retained as an additional basis-dependent reconstruction associated with the ARP framework.

3.1.5 Pipeline Configurations

Combining the two basis-construction methods with the three column-selection methods produces **six pipeline configurations**. Each configuration follows the same overall CSSP procedure described in Section 3.1.1, while differing only in the method used to construct the basis or select the columns.

Pipeline	Basis Construction	Column Selection
Deterministic SVD + Osinsky	Deterministic SVD	Osinsky
Deterministic SVD + ARP	Deterministic SVD	ARP
Deterministic SVD + Rejection ARP	Deterministic SVD	Rejection ARP
Randomized SVD + Osinsky	Randomized SVD	Osinsky
Randomized SVD + ARP	Randomized SVD	ARP
Randomized SVD + Rejection ARP	Randomized SVD	Rejection ARP

Table 3.1: CSSP Pipeline Configurations

The pipeline implementation was designed so that basis construction and column selection could be varied independently. When multiple selection methods are compared using deterministic SVD, the same deterministic basis is used for each selector. Similarly, for comparisons involving randomized SVD, a single randomized basis generated for a particular run is shared among the corresponding selection methods. This prevents differences in the generated basis from being unintentionally attributed to differences between the column-selection algorithms.

The six configurations also support controlled comparisons of where randomization is introduced. Comparing deterministic SVD + Osinsky with deterministic SVD + ARP or Rejection ARP changes the column-selection method while retaining the same deterministic basis. Conversely, comparing deterministic and randomized SVD while keeping the same selector examines the effect of randomization during basis construction. The complete set of pipelines therefore enables randomization at the basis-construction and column-selection stages to be studied separately as well as in combination.

3.2 Implementation Validation

3.2.1 Input and Numerical Validation

A common validation layer was implemented to ensure that invalid inputs and numerically unsuitable intermediate values were detected before they propagated through the CSSP pipeline. This validation was applied across the basis-construction, column-selection, reconstruction, and evaluation components.

Input matrices were required to be non-empty, two-dimensional, real-valued, and finite. The target rank k was checked to ensure that it was an integer within the valid range for the supplied matrix. Parameters specific to randomized algorithms, including oversampling, the number of power iterations, random seeds, and block sizes where applicable, were also checked before execution.

Additional validation was performed on intermediate quantities used by the pipeline. Selected column indices were required to be one-dimensional, integer-valued, distinct, and within the valid column range. Basis matrices were checked for dimensional compatibility with the input matrix and for numerical orthonormality. For reconstruction methods that depend on the selected rows of the basis, the corresponding k\times k submatrix was checked for numerical nonsingularity before the reconstruction was performed.

The validation layer also checks the outputs used during evaluation. Reconstructed matrices must have the same dimensions as the original matrix, while computed error quantities and optimal approximation baselines are required to be finite and non-negative where mathematically defined. These checks were designed to distinguish invalid inputs or numerical failures from genuine differences in algorithmic performance, thereby reducing the possibility of misleading experimental results.

3.2.2 Algorithm Validation Against Independent References

The core algorithms were validated using independent reference calculations rather than relying only on outputs produced by the main pipeline. This was intended to detect implementation errors that could otherwise remain hidden if the same computational logic were reused in both the algorithm and its tests.

The Osinsky implementation was compared with an independent Python translation of the reference MATLAB procedure. For controlled input matrices, the selected column indices produced by the implementation were compared with those obtained from the reference procedure. This provided a direct check of the deterministic selection logic.

ARP was validated at the individual selection-step level. The adaptive sampling probabilities generated during the algorithm were compared with probabilities calculated independently using a direct residual-projection formulation. This checks whether the probability distribution is updated as intended as successive pivots are selected, rather than validating only the final selected subset.

Randomized SVD was similarly checked using an independently constructed sequence consisting of Gaussian sketch generation, power iterations, orthonormalization, and SVD of the projected matrix. The resulting subspace was compared with that produced by the implemented randomized SVD procedure. Reconstruction operations and approximation-error calculations were also checked independently against their corresponding mathematical definitions.

Together, these tests provide evidence that the principal computational components follow their intended algorithms and mathematical formulations. They complement the input and numerical checks by testing the actual computational behavior of the implemented methods.

3.2.3 Randomized Algorithm Validation

Additional validation was performed for ARP, Rejection ARP, and randomized SVD because randomized algorithms cannot be assessed solely by comparing the output of a single execution. The tests therefore considered both reproducibility under controlled seeds and the statistical behavior of the randomized selection procedures.

Fixed random seeds were used to verify reproducibility. Repeated executions with the same input and seed were checked for consistent results, while the implementations also allowed different seeds to be supplied for subsequent experimental trials. For the randomized column-selection algorithms, additional checks ensured that exactly k distinct and valid indices were selected and that the selected basis rows satisfied the numerical requirements needed for reconstruction.

The sampling behavior of ARP and Rejection ARP was examined on a small controlled orthonormal basis for which the theoretical probability of selecting each subset J can be

calculated. For a k-element subset, the target distribution is proportional to $\det(V_J)^2$, where V_J denotes the $k \times k$ submatrix formed from the selected rows of the basis. Each randomized selector was executed repeatedly, and the empirical frequencies of the possible subsets were compared with these theoretical probabilities. This provides a statistical check that the implementations reproduce the intended subset-sampling behavior rather than merely returning valid column indices.

These validation tests are distinct from the later experiments on randomization variability. Their purpose is to check the implementation of the randomized mechanisms themselves, whereas the experimental analysis examines how variation between randomized runs affects practical CSSP approximation quality and computational performance.

3.2.4 Numerical Robustness and Edge Cases

Numerical robustness was tested separately from the algorithmic validation to ensure that the implementations remained reliable for matrices with challenging numerical properties. Particular attention was given to the deterministic Osinsky implementation because its selection criterion involves quantities whose magnitudes can vary considerably. Tests were performed using matrices scaled by very small and very large constants, and the implementation uses scale normalization to reduce the possibility of numerical overflow or underflow during selection.

Edge cases were also included to verify expected behavior for inputs such as zero matrices, exact low-rank matrices, and cases where the optimal rank-k approximation error is zero or extremely small. The implementations were additionally checked to ensure that input matrices were not unintentionally modified during computation.

The calculation of the optimal rank-k SVD error received particular attention because it is used as a baseline in the experimental evaluation. The optimal Frobenius-norm error was computed directly from the trailing singular values,

$$E_{\mathrm{SVD},k} = \left(\sum_{i=k+1}^{r} \sigma_i^2 \right)^{1/2}$$

where r is the number of available singular values. This avoids numerical cancellation that can occur when the residual error is obtained by subtracting two nearly equal squared norms. When the optimal error is exactly zero, ratios involving this quantity are treated as undefined rather than reporting a misleading finite value.

These tests were used to verify that numerical edge cases and floating-point effects were handled explicitly and did not silently produce invalid approximation metrics or column selections.

3.2.5 Integration and Test Suite

In addition to validating individual algorithms and numerical operations, integration tests were used to verify that the components functioned correctly when combined into the complete CSSP pipeline. These tests covered the interaction between basis construction, column selection, reconstruction, and evaluation, including all six standard pipeline configurations.

The integration tests verified properties such as the dimensions and validity of generated bases, the number and uniqueness of selected column indices, compatibility between selected columns and reconstruction methods, and the dimensions and finiteness of the resulting approximations and error metrics. Tests also verified the shared-basis design used for controlled comparisons, ensuring that different selectors receive the same deterministic or randomized basis when required. Fixed-seed tests were used throughout randomized components to support reproducible execution.

Additional tests covered the experimental framework itself, including dataset loading, reconstruction timing, optimal-error computation, runtime experiments, matrix-structure experiments, sparsity comparisons, randomization-variability experiments, and the separate Rejection ARP benchmark. This was important because errors in experimental scripts or metric calculations could otherwise affect the conclusions even when the underlying algorithms were implemented correctly.

At the completion of the implementation and experimental framework, the automated test suite contained 576 passing pytest cases collected from 14 test files. Many tests were parameterized, meaning that a single test function generated multiple independently executed test cases. Table 3.2 summarizes the distribution and primary purpose of these cases.

Test File	Cases	Primary Purpose
`test_validation.py`	374	Reject invalid matrices, ranks, parameters, bases, and indices
`test_osinsky_scaling.py`	31	Check Osinsky selection and numerical scale robustness
`test_dataset_loaders.py`	29	Validate GEO and sparse dataset parsing
`test_optimal_error.py`	26	Check exact SVD rank-k error baselines
`test_evaluation.py`	20	Check approximation errors and optimality ratios

`test_reconstruction_timing.py`	15	Verify reconstruction timing and timing accounting
`test_runtime_experiment.py`	15	Validate runtime experiment design
`test_reference_algorithms.py`	14	Compare implementations against independent formulations
`test_sparsity_experiment.py`	12	Validate dense/CSR sparsity comparisons
`test_rejection_benchmark.py`	10	Validate Rejection ARP benchmark behavior
`test_structure_experiment.py`	10	Validate matrix-structure experiments
`test_variability_experiment.py`	10	Validate randomization-variability experiment design
`test_accuracy_experiment.py`	4	Validate accuracy experiment logic
`test_pipelines.py`	6	Check integration of the core CSSP pipelines

Table 3.2: Summary of Automated Validation Test Suite



Fig 3.1: Terminal output showing successful execution of the complete automated test suite of 576 pytest cases

The successful execution of these tests provides evidence that the implementation behaves consistently with the reference procedures, mathematical properties, numerical requirements, and experimental design checks examined during development. The test suite is therefore treated as implementation and experimental-framework validation rather than as a formal proof of algorithmic correctness.

3.3 Datasets and Test Matrices

The experimental evaluation used a combination of controlled synthetic matrices, real-world datasets, and large synthetic benchmark matrices. The synthetic matrices were used to isolate specific numerical properties such as rank structure, noise, singular-value decay, and sparsity, while the real-world datasets provided matrices with naturally occurring structure. Separate large-scale matrices were used for the Rejection ARP runtime benchmark.

3.3.1 Synthetic Test Matrices

A low-rank-plus-noise matrix was used as a controlled approximately low-rank test case. It was constructed as:

$$A = U\Sigma V^T + 0.02N$$

where U and V are orthonormal factors, the signal rank is 25, the singular values decay geometrically from 20 to 1, and N contains independent Gaussian entries. The default matrix size was 160 x 120.

To investigate the effect of matrix structure, six additional 500 x 500 matrix families were generated: Gaussian, exact low-rank, low-rank with small noise, low-rank with large noise, rapid singular-value decay, and slow singular-value decay. These matrices provide different spectral and rank characteristics while maintaining the same dimensions. They were normalized to unit Frobenius norm before evaluation.

Matched dense and sparse matrices were also generated at densities of 100%, 50%, 10%, and 1%. Each matrix had dimensions 500 x 500, and exactly the same numerical values were represented in both dense and Compressed Sparse Row (CSR) formats. This provides paired inputs for examining the effect of sparse representation without changing the underlying matrix.

3.3.2 Real-World Datasets

The GSE10072 gene-expression dataset from the NCBI Gene Expression Omnibus was used as a tall real-world matrix. It contains 22,283 probe measurements across 107 tissue samples, resulting in a matrix of size 22,283 x 107. In the context of CSSP, the columns correspond to actual tissue samples, giving a direct interpretation to the selected columns.

A second real-world dataset was the `Meszaros/large` matrix from the SuiteSparse Matrix Collection. The original matrix has dimensions 4,282 x 8,617 and approximately 20,635 non-zero entries. For evaluation with the standard CSSP pipelines, a reproducibly sampled set of 400 non-zero columns was used, followed by removal of rows that were zero within the resulting submatrix. The reduced matrix was then converted to dense form, while the original column indices were retained.

3.3.3 Rejection ARP Benchmark Matrices

Two additional synthetic matrices were used specifically for large-scale benchmarking of ARP and Rejection ARP. These matrices followed the reference-style construction associated with the Rejection ARP work and were kept separate from the real-world datasets.

The dense benchmark matrix had dimensions 10,000 x 10,000. The sparse benchmark matrix had an original size of 1,000,000 x 10,000, with 30 non-zero entries per column. Both matrices used Gaussian values with row-dependent i^{-2} scaling. For column subset selection, the benchmark matrices were operated on in transposed orientation. The sparse matrix remained sparse throughout the benchmark to avoid the memory cost of constructing a dense representation.

Matrix / Dataset	Dimensions	Category
Low-rank+noise	160×120	Synthetic
Structure families	500×500	Synthetic
Dense/CSR matrices	500×500	Synthetic
GSE10072 (DNA) dataset	$22,283 \times 107$	Real-world
Meszaros dataset	$4,282 \times 8,617$	Real-world
Dense benchmark	$10,000 \times 10,000$	Synthetic benchmark
Sparse benchmark	$1,000,000 \times 10,000$	Synthetic benchmark

Table 3.3: Summary of datasets and test matrices used.

3.4 Experimental Design

3.4.1 Evaluation Metrics and Baselines

The algorithms were evaluated using approximation-quality and computational-performance measures. Relative errors were used as the primary accuracy measures because the experiments involved matrices with different dimensions and numerical scales. For an input matrix A and its reconstruction $\widehat{A}$, the relative Frobenius-norm error was calculated as follows:

$$E_{\text{rel},F} = \frac{\left\| A - \widehat{A} \right\|_F}{\left\| A \right\|_F}$$

Absolute Frobenius error was also retained where required. Frobenius-norm measures formed the primary accuracy outputs of the experiments, while absolute and relative spectral-norm errors were also implemented for additional evaluation.

To compare reconstruction quality with the best unrestricted rank-k approximation, the exact SVD error was used as a baseline:

$$E_{\mathrm{SVD},k} = \left(\sum_{i=k+1}^{r} \sigma_i^2 \right)^{1/2}$$

where σ_i are the singular values of A and $r = min(m,n)$. This baseline was computed from the complete exact singular-value spectrum rather than from a randomized basis. The corresponding SVD optimality ratio was

$$R_{\mathrm{SVD}} = \frac{\left\| A - \widehat{A} \right\|_F}{E_{\mathrm{SVD},k}}$$

A value close to 1 indicates reconstruction quality close to the best unrestricted rank-k approximation. This is not, however, the CSSP optimum, since CSSP is restricted to selecting actual columns of A. When the optimal SVD error is exactly zero, the ratio is undefined and is reported as `NaN`.

For the exact small-scale CSSP comparison, a separate baseline was obtained by exhaustively evaluating all subsets J of k columns:

$$E_{\mathrm{CSSP\text{-}opt}} = \min_{|J|=k} \left\| A - A_{:,J} A^{+}_{:,J} A \right\|_F$$

The selected subset was then compared with this optimum using:

$$R_{\mathrm{CSSP}} = \frac{E_{\mathrm{alg}}}{E_{\mathrm{CSSP\text{-}opt}}}$$

Thus, the SVD baseline measures performance against the best unrestricted rank-k approximation, whereas the exhaustive CSSP baseline measures performance against the best possible selection of k actual columns.

Computational performance was evaluated using execution time, with basis construction, column selection, and reconstruction or interpolation timed separately where applicable. Baseline computations and other evaluation operations were excluded from the corresponding algorithm timings. Additional measurements, such as storage requirements and Rejection ARP sampling diagnostics, were recorded only in experiments where they were relevant.

3.4.2 Experimental Controls and Reproducibility

Fixed random seeds were used for reproducible synthetic data generation and randomized algorithms. Where variability was being investigated, randomized methods were repeated across multiple seeds, while deterministic methods were used as fixed reference cases. For controlled accuracy comparisons, computational components were shared where appropriate. The selector-variability experiment similarly fixed the matrix and exact SVD basis while varying only the selector seed. Dense and CSR comparisons used identical matrix values and paired random seeds to ensure that the underlying numerical problem remained unchanged.

Runtime experiments were conducted independently from these accuracy comparisons. Each timed pipeline constructed its own basis and used warm-up runs, repeated measurements, shuffled execution order, and controlled BLAS threading. Basis construction, column selection, and reconstruction times were recorded separately where applicable, with evaluation and baseline calculations excluded from the corresponding algorithm-time measurements.

3.4.3 Experiments Conducted

Eight experiments were designed to evaluate different aspects of the deterministic and randomized CSSP methods. They examine approximation accuracy, randomization variability, computational scalability, matrix structure, sparse representation, and performance on real-world data. The experiments also compare the complete CSSP pipelines to study the effects of randomization in both basis construction and column selection. Where possible, each experiment isolates a specific factor while following the common metrics and controls described in Sections 3.4.1 and 3.4.2.

E1: Exact CSSP Comparison

A small 8×10 Gaussian matrix with k=3 was used to determine how closely the selection methods approach the true CSSP optimum. All 120 possible three-column subsets were enumerated to obtain the minimum orthogonal reconstruction error. Osinsky, ARP, and Rejection ARP were then evaluated using a shared exact SVD basis, with repeated trials for the randomized selectors. Their errors were compared against both the exact CSSP optimum and the optimal rank-k SVD baseline.

E2: Approximation Accuracy vs. Target Rank

The effect of the selected-column count was investigated using $k \in \{5, 10, 20, 30, 40\}$. All six pipeline configurations were evaluated on the noisy low-rank synthetic matrix and the GSE10072 gene-expression matrix. Randomized configurations were evaluated over repeated trials to capture variation in approximation quality as k increased.

E3: Randomization Variability

The variability introduced specifically by randomized column selection was examined by holding the input matrix and exact SVD basis fixed. With k=10, ARP and Rejection ARP were each repeated over 100 trials and compared with the fixed deterministic Osinsky result. This design isolates selector randomness from variation caused by randomized basis construction.

E4: Runtime and Scalability

Computational scalability was evaluated on square synthetic matrices of increasing size: 100, 250, 500, and 1000, with k=10. All six pipelines were timed across repeated executions, with basis construction, column selection, and reconstruction measured separately. A separate large-scale benchmark compared ARP and Rejection ARP on the $10,000 \times 10,000$ dense benchmark and the transposed $10,000 \times 1,000,000$ sparse benchmark using a factorized workflow that avoided explicitly forming the full approximation.

E5: Effect of Matrix Structure

To examine sensitivity to mathematical structure independently of matrix dimensions, six 500×500 matrix families were evaluated: Gaussian, exact low-rank, low-rank with small noise, low-rank with large noise, rapid singular-value decay, and slow singular-value decay. The selected-column count was fixed at k=10, and all six pipelines were compared across multiple independently generated matrix instances.

E6: Dense vs. Sparse Representation

The effect of sparse representation was investigated using matched 500×500 matrices at densities of 100%, 50%, 10%, and 1%. Each numerical matrix was represented in both dense and CSR formats, allowing a paired comparison without changing the underlying values. ARP and Rejection ARP were evaluated using the sparse-aware, factorized workflow, with runtime, approximation quality, and numerical storage requirements compared between representations.

E7: Real-World Evaluation

The methods were evaluated on the GSE10072 gene-expression dataset and the SuiteSparse `Meszaros/large` matrix to examine their behavior on matrices arising from real applications. The SuiteSparse evaluation used the reproducibly sampled 400-column submatrix described in Section 3.3. Approximation quality and computational performance were evaluated while retaining mappings from selected columns to their original dataset identifiers.

E8: Full Pipeline Comparison

The final experiment compared the six combinations of deterministic or randomized SVD with Osinsky, ARP, or Rejection ARP. Using real-world inputs considered in the preceding experiments, the comparison examined approximation quality, randomized variability, and computational cost. Controlled comparisons were used to distinguish the effect of randomizing basis construction from the effect of randomizing column selection.

Experiment	Main Question	Main Variable	Primary Measures
E1	How close are selected columns to the true CSSP optimum?	Selection method	CSSP error / exact optimum
E2	How does accuracy change with k?	Target rank	Approximation error
E3	How much variability does randomized selection introduce?	Random seed	Error distribution
E4	How does computational cost scale?	Matrix size	Runtime
E5	How does matrix structure affect performance?	Matrix structure	Error, runtime
E6	How does sparse representation affect the methods?	Storage representation (dense/sparse)	Error, runtime, storage
E7	How do the methods behave on real data?	Dataset	Error, runtime, variability
E8	What changes when randomization is introduced at different pipeline stages?	Basis + selector	Error and runtime

Table 3.4: Summary of Experimental Design

4. Results and Discussion

4.1 Exact CSSP Comparison

The following figure presents the terminal output from the exact CSSP comparison, summarizing the exhaustive baseline and the results of the three column-selection methods.

```
(.venv) sathsaraniamarasinghe@Sasas-MacBook-Air CSSP % python experiments/run_exact_
cssp_experiment.py
E1: Gaussian 8 x 10, k=3, data seed=42
Enumerated subsets: 120
SVD baseline: 3.462925157
Best CSSP error: 3.762732429; one minimizing subset (zero-based): [0, 4, 7]

Method                Runs      Mean error        Std error     Mean CSSP ratio
exact_cssp               1       3.7627324                 0                   1
osinsky                  1       3.7627324                 0                   1
arp                    100       4.1297931       0.30928811           1.0975516
  Best observed: error=3.7627324, CSSP ratio=1, seed=53, indices=[0, 7, 4]
rejection_arp          100        4.144813       0.30167799           1.1015434
  Best observed: error=3.7627324, CSSP ratio=1, seed=3, indices=[0, 7, 4]

Ordering checked with numerical tolerance: SVD <= best CSSP <= method error.
Best observed randomized results are not single-run guarantees.
```

Fig 4.1: Exact CSSP comparison of Osinsky, ARP, and Rejection ARP against the exhaustive optimum.

The results confirm the expected relationship between the optimal SVD approximation, the exhaustive CSSP optimum, and the subsets produced by the selection methods. Osinsky reached the exact CSSP optimum for this test case, while ARP and Rejection ARP showed variability across repeated trials but were also capable of reaching an optimal subset. Overall, the experiment provides a small-scale validation of the three column-selection implementations and demonstrates the distinction between deterministic selection and the variability introduced by randomized selection.

4.2 Accuracy vs. Target Rank

The figures below show how the relative reconstruction error changes as the number of selected columns k increases for the six CSSP pipelines on the DNA and synthetic datasets.

Across both datasets, the relative reconstruction error decreases as k increases for all six pipelines. This is expected because selecting more columns provides a larger set of information for reconstructing the original matrix, allowing the approximation to capture more of its structure. The six pipelines follow a similar decreasing trend, although their error levels and variability differ slightly.

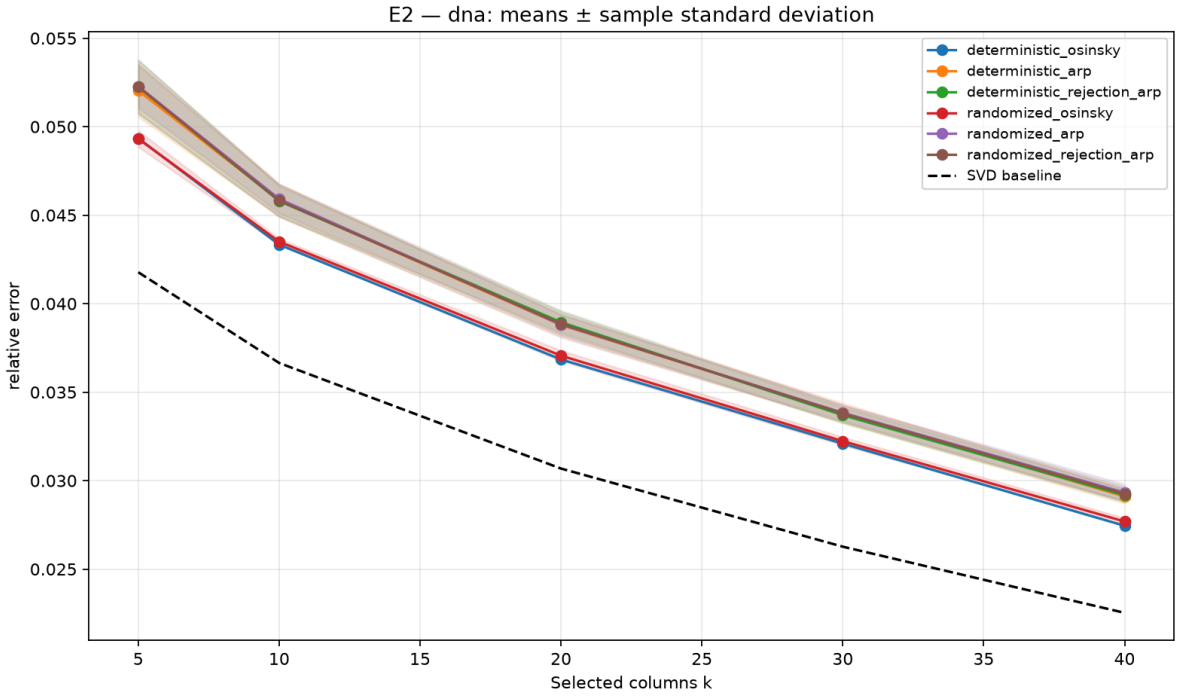


Fig. 4.2: Relative reconstruction error versus selected columns for the DNA dataset.

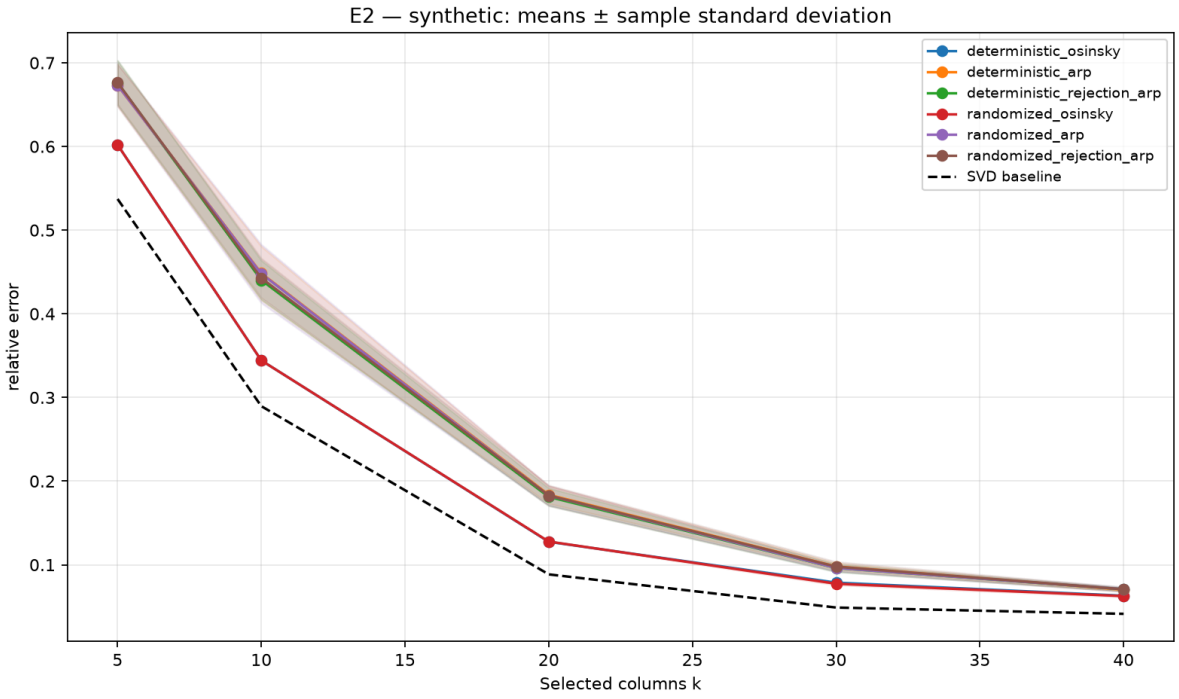


Fig. 4.3: Relative reconstruction error versus selected columns for the synthetic dataset.

4.3 Randomization Variability

Figures 4.5 and 4.6 show the distribution of relative reconstruction errors for ARP and Rejection ARP across repeated random trials, with deterministic Osinsky used as the fixed reference. The terminal output in Fig. 4.4 summarizes the results over 100 trials for each randomized selector.

The results show that both randomized selectors produced higher relative reconstruction errors than deterministic Osinsky in all 100 trials on both datasets. ARP achieved a slightly lower mean error than Rejection ARP on both the synthetic and DNA datasets. The variation across trials also demonstrates the effect of the random seed on the selected columns and resulting reconstruction accuracy, whereas Osinsky remains fixed for the same input and basis.

```
synthetic osinsky: relative error mean=0.344282, std=0; better/tied/worse=0/1/0
synthetic arp: relative error mean=0.443844, std=0.0263701; better/tied/worse=0/0/100
synthetic rejection_arp: relative error mean=0.448148, std=0.027522; better/tied/worse=0/0/100
dna osinsky: relative error mean=0.0433338, std=0; better/tied/worse=0/1/0
dna arp: relative error mean=0.045839, std=0.000995558; better/tied/worse=0/0/100
dna rejection_arp: relative error mean=0.0457762, std=0.000858256; better/tied/worse=0/0/100
Saved 402 measurements to /Users/sathsaraniamarasinghe/CSSP/results/e3/variability_full
```

Fig. 4.4: Terminal output of summary of relative reconstruction errors across 100 randomized trials.

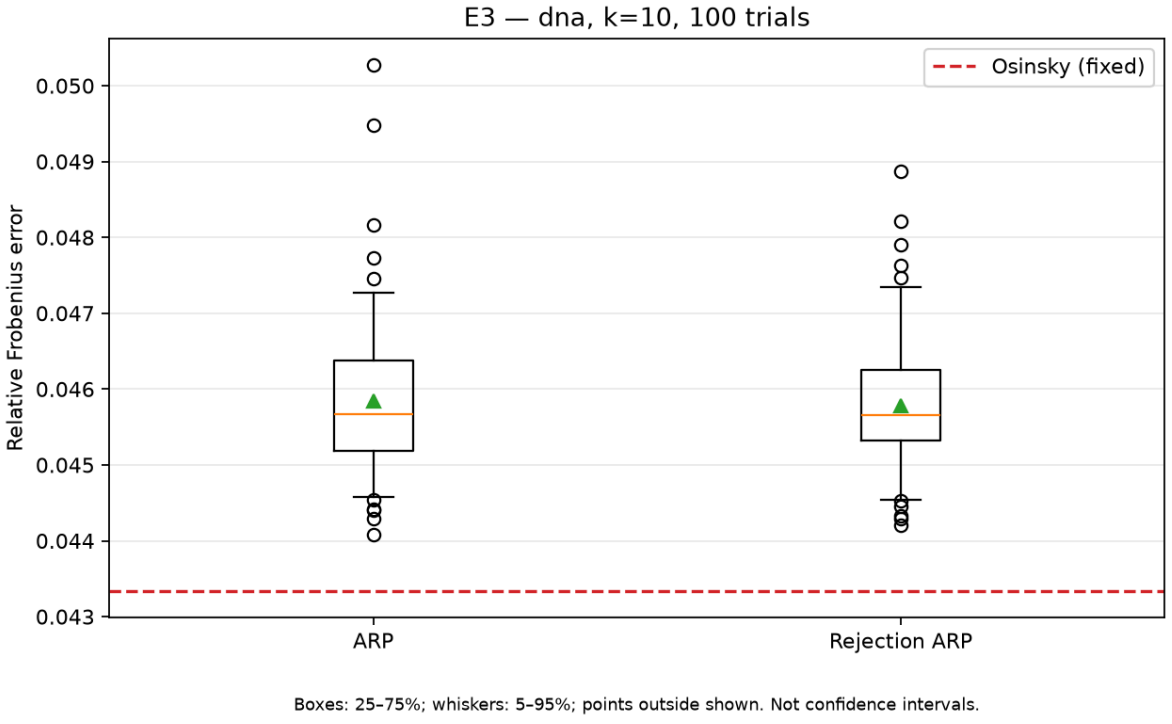


Fig. 4.5: Randomization variability of ARP and Rejection ARP on the DNA dataset.

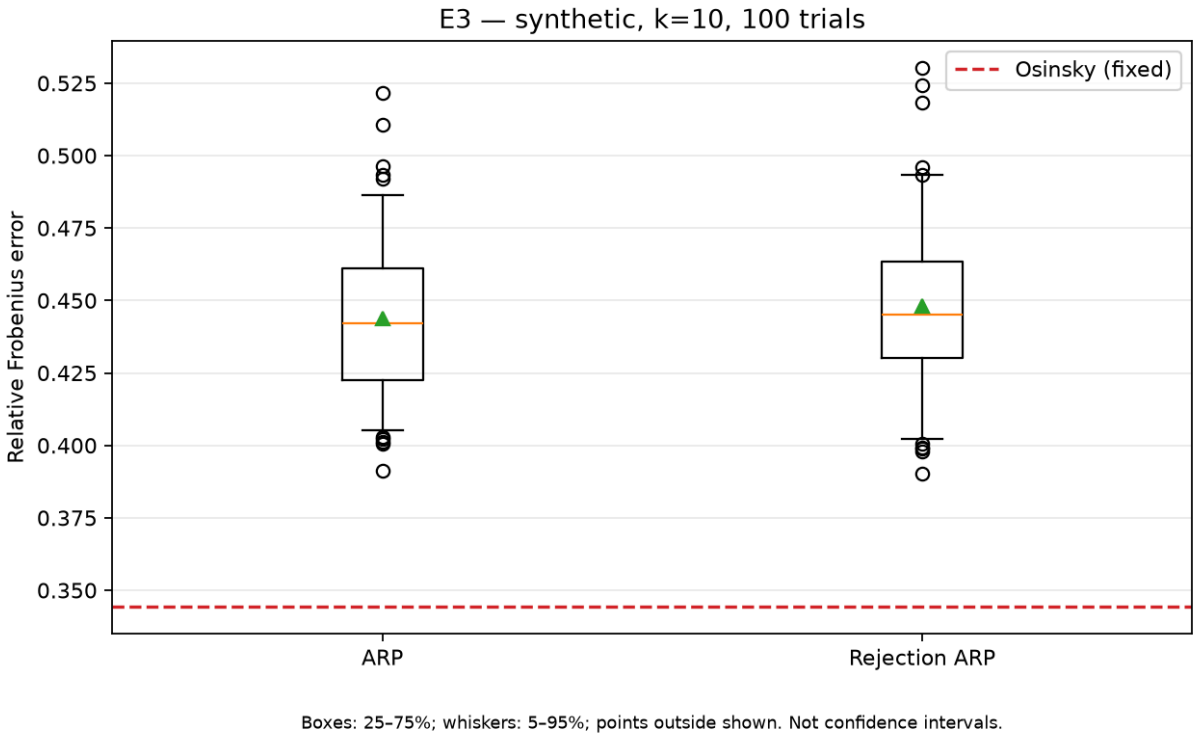


Fig. 4.6: Randomization variability of ARP and Rejection ARP on the synthetic dataset.

4.4 Runtime and Scalability

The following figures show how the median algorithm runtime of the six CSSP pipelines changes as the matrix dimension increases.

Size	Pipeline	Basis	Selection	Recon.	Total
Median times in milliseconds					
100×100	deterministic_arp	1.509	0.536	0.187	2.214
100×100	deterministic_osinsky	1.585	0.806	0.187	2.722
100×100	randomized_arp	0.452	0.524	0.181	1.158
100×100	randomized_osinsky	0.443	0.766	0.177	1.387
250×250	deterministic_arp	10.104	0.707	0.753	11.639
250×250	deterministic_osinsky	10.043	1.883	0.740	12.699
250×250	randomized_arp	0.819	0.633	0.720	2.192
250×250	randomized_osinsky	0.834	1.822	0.737	3.407
500×500	deterministic_arp	42.261	0.961	2.275	45.487
500×500	deterministic_osinsky	42.367	5.438	2.252	50.087
500×500	randomized_arp	1.598	0.837	2.238	4.665
500×500	randomized_osinsky	1.611	5.289	2.242	9.124
1000×1000	deterministic_arp	198.721	1.433	9.246	209.412
1000×1000	deterministic_osinsky	197.256	20.040	8.693	226.221
1000×1000	randomized_arp	4.384	1.297	8.703	14.429
1000×1000	randomized_osinsky	4.381	19.569	8.839	32.661

Fig. 4.7: Stage-wise runtime measurements for the CSSP pipelines.

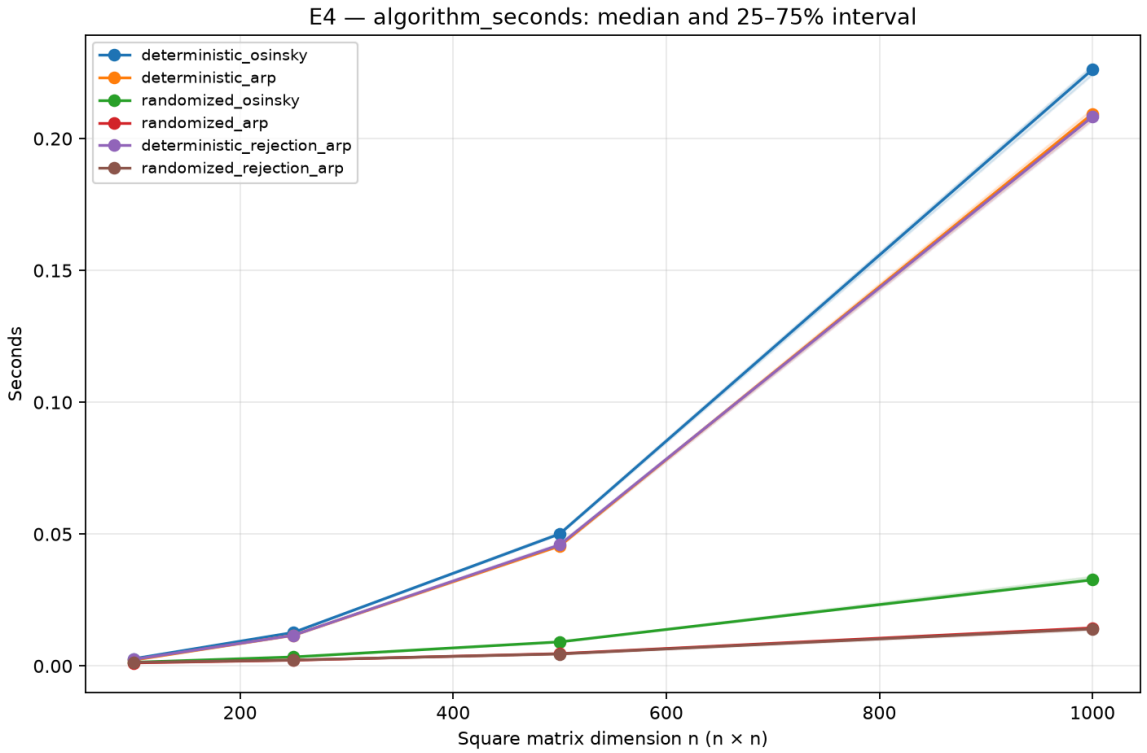


Fig. 4.8: Runtime scaling of the six CSSP pipelines with increasing matrix dimension.

Runtime increased with matrix size for all six pipelines, but the largest difference was caused by the basis-construction method. Pipelines using deterministic SVD became considerably more expensive as the matrix dimension increased, whereas randomized SVD substantially reduced the overall runtime. Differences between Osinsky, ARP, and Rejection ARP were comparatively small at these matrix sizes, indicating that basis construction dominated the total computational cost. The performance differences between sequential ARP and Rejection ARP become more relevant for much larger problems, which are examined separately in the large-scale benchmark.

4.5 Effect of Matrix Structure

The following figures show how different matrix structures affect the runtime and relative reconstruction error of the six CSSP pipelines.

The results show that matrix structure has a much stronger effect on reconstruction accuracy than on runtime. The Gaussian matrix produced the highest relative errors, while the exact low-rank, small-noise, and slow-decay matrices were reconstructed with substantially lower errors. In contrast, runtime remained relatively consistent across the different structures for each pipeline. The randomized-SVD pipelines were consistently faster than their deterministic-SVD counterparts, while the accuracy differences between the six pipelines depended on the matrix structure.

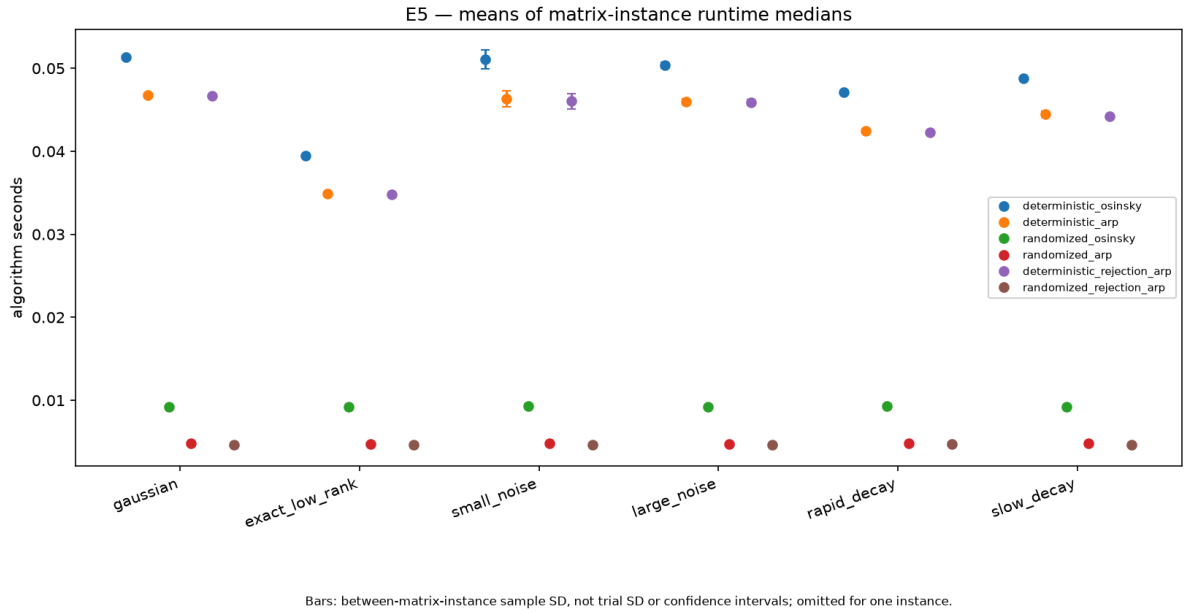


Fig. 4.9: Algorithm runtime of the six CSSP pipelines across different matrix structures.

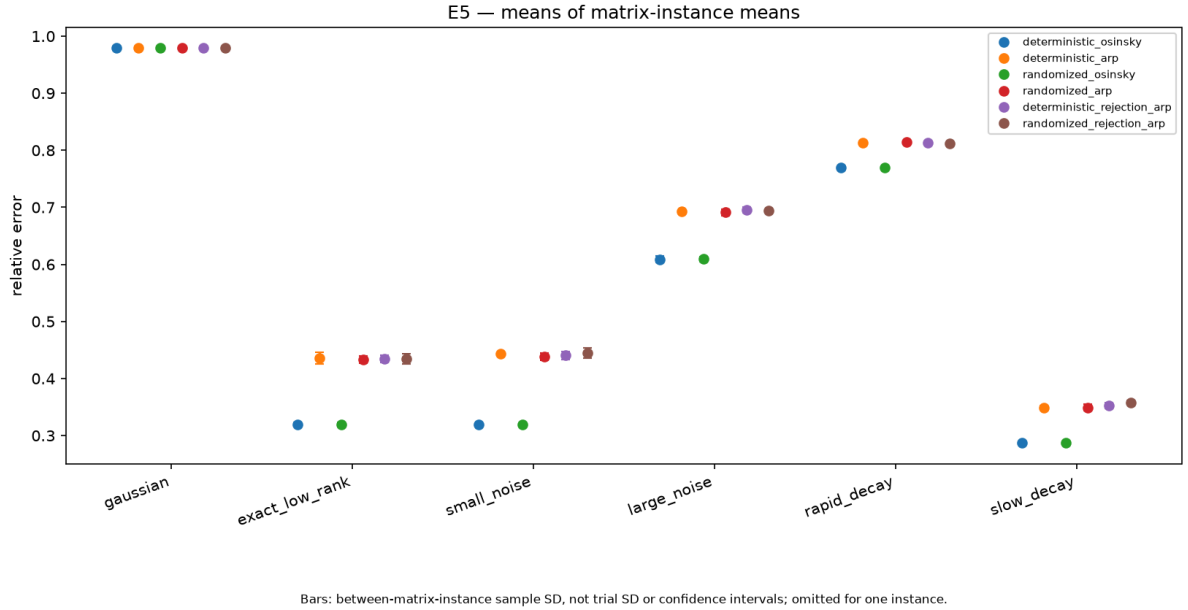


Fig. 4.10: Relative reconstruction error of the six CSSP pipelines across different matrix structures.

4.6 Dense and Sparse Performance

The below figure compares the runtime of ARP and Rejection ARP using matched dense and CSR representations as the percentage of non-zero entries changes.

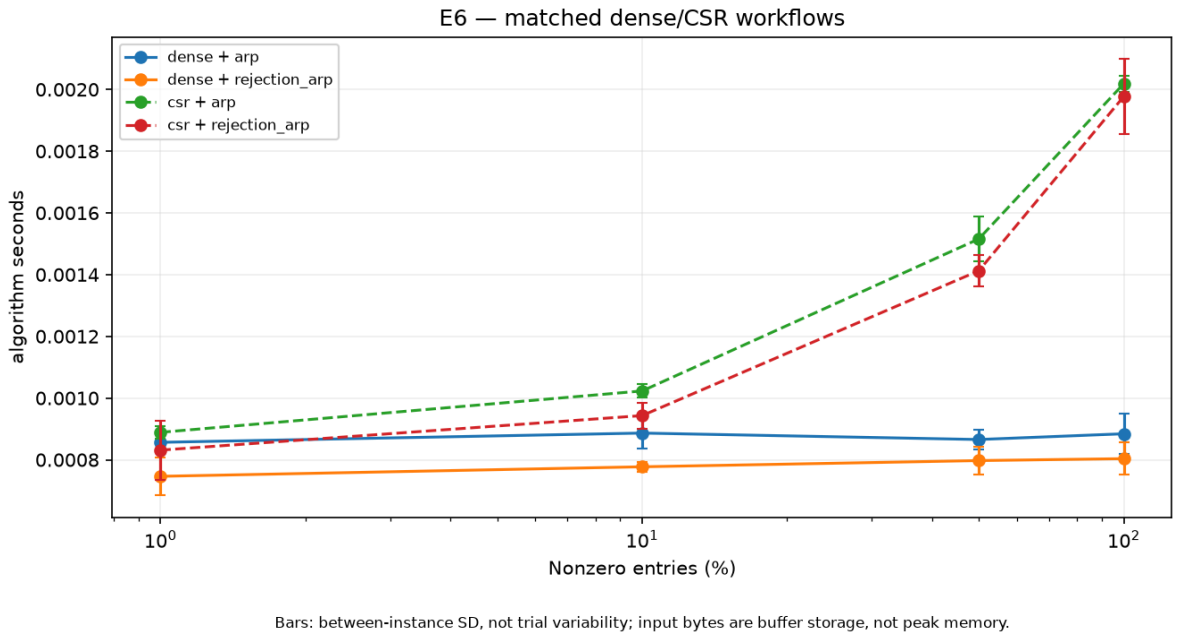


Fig. 4.11: Runtime comparison of dense and CSR workflows at different matrix densities.

The results show that dense execution time remains relatively stable as density changes, whereas CSR runtime increases as the matrix becomes denser. At low densities, the sparse representation provides comparable performance, but its advantage decreases as more nonzero entries must be processed. Rejection ARP is slightly faster than sequential ARP for both representations in the observed results. Overall, sparse representation is most useful for highly sparse matrices, while dense storage becomes more competitive as matrix density increases.

4.7 Real-World Dataset Results

On the DNA dataset, the Osinsky-based pipelines achieved lower reconstruction errors than the ARP and Rejection ARP pipelines, while randomized SVD reduced runtime compared with deterministic SVD. On the Meszaros subset, all six pipelines produced essentially the same reconstruction error and were very close to the optimal SVD baseline, while the randomized-SVD pipelines again required less time. Overall, the results show that the effect of the selection method depends on the dataset, while randomized basis construction provided a clear runtime reduction on both real-world datasets.

```
(.venv) sathsaraniamarasinghe@Sasas-MacBook-Air CSSP % python experiments/run_dna_experiment.py --pipelines 6 --run-name dna_full

GSE10072: (22283, 107), 107 tissue samples
dna: k=10, trial 1/10 complete
dna: k=10, trial 2/10 complete
dna: k=10, trial 3/10 complete
dna: k=10, trial 4/10 complete
dna: k=10, trial 5/10 complete
dna: k=10, trial 6/10 complete
dna: k=10, trial 7/10 complete
dna: k=10, trial 8/10 complete
dna: k=10, trial 9/10 complete
dna: k=10, trial 10/10 complete

   k Pipeline                      Mean rel. error   Mean SVD ratio   Median alg. ms
  10 deterministic_osinsky               0.0433338          1.18243          128.182
  10 deterministic_arp                   0.0457265          1.24772           92.689
  10 randomized_osinsky                  0.0434114          1.18455           74.401
  10 randomized_arp                      0.0454989          1.24151           37.750
  10 deterministic_rejection_arp         0.0458694          1.25162           93.298
  10 randomized_rejection_arp            0.0458094          1.24998           37.968
Saved 60 measurements to /Users/sathsaraniamarasinghe/CSSP/results/e7/dna/dna_full
```

Fig. 4.12: Accuracy and runtime results for the six CSSP pipelines on the DNA dataset.

```
(.venv) sathsaraniamarasinghe@Sasas-MacBook-Air CSSP % python experiments/run_large_experiment.py \
  --pipelines 6 --columns 400 --data-seed 42 \
  --ks 10 20 40 --trials 10 \
  --run-name meszaros_rank_sweep
Meszaros/large: original (4282, 8617), dense subset (949, 400)
meszaros: k=10, trial 1/10 complete
meszaros: k=10, trial 2/10 complete
meszaros: k=10, trial 3/10 complete
meszaros: k=10, trial 4/10 complete
meszaros: k=10, trial 5/10 complete
meszaros: k=10, trial 6/10 complete
meszaros: k=10, trial 7/10 complete
meszaros: k=10, trial 8/10 complete
meszaros: k=10, trial 9/10 complete
meszaros: k=10, trial 10/10 complete
meszaros: k=20, trial 1/10 complete
meszaros: k=20, trial 2/10 complete
meszaros: k=20, trial 3/10 complete
meszaros: k=20, trial 4/10 complete
meszaros: k=20, trial 5/10 complete
meszaros: k=20, trial 6/10 complete
meszaros: k=20, trial 7/10 complete
meszaros: k=20, trial 8/10 complete
meszaros: k=20, trial 9/10 complete
meszaros: k=20, trial 10/10 complete
meszaros: k=40, trial 1/10 complete
meszaros: k=40, trial 2/10 complete
meszaros: k=40, trial 3/10 complete
meszaros: k=40, trial 4/10 complete
meszaros: k=40, trial 5/10 complete
meszaros: k=40, trial 6/10 complete
meszaros: k=40, trial 7/10 complete
meszaros: k=40, trial 8/10 complete
meszaros: k=40, trial 9/10 complete
meszaros: k=40, trial 10/10 complete

    k Pipeline                      Mean rel. error   Mean SVD ratio   Median alg. ms
   10 deterministic_osinsky              0.197161                1          64.686
   10 deterministic_arp                  0.197161                1          61.843
   10 randomized_osinsky                 0.197161                1          17.769
   10 randomized_arp                     0.197161                1           9.405
   10 deterministic_rejection_arp        0.197161                1          68.215
   10 randomized_rejection_arp           0.197161                1          10.362
   20 deterministic_osinsky           0.00372962          1.00001          62.385
   20 deterministic_arp               0.00372962          1.00001          46.600
   20 randomized_osinsky              0.00372962          1.00001          24.155
   20 randomized_arp                  0.00372962          1.00001          10.307
   20 deterministic_rejection_arp     0.00372962          1.00001          48.300
   20 randomized_rejection_arp        0.00372962          1.00001           9.716
   40 deterministic_osinsky           0.00275785           1.0011         124.767
   40 deterministic_arp               0.00275905          1.00153          71.285
   40 randomized_osinsky              0.00276022          1.00196          52.138
   40 randomized_arp                  0.00277357           1.0068          27.054
   40 deterministic_rejection_arp     0.00276727          1.00451          75.888
   40 randomized_rejection_arp        0.00277526          1.00741          24.412
```

Fig. 4.13: Accuracy and runtime results for the six CSSP pipelines on the Meszaros dataset.

4.8 Full Pipeline Comparison

Accuracy–runtime trade-off plots were produced for both real-world datasets using the measurements obtained in E7. Figure 4.14 presents the DNA result, showing the relationship between reconstruction accuracy and algorithm runtime for all six pipelines.

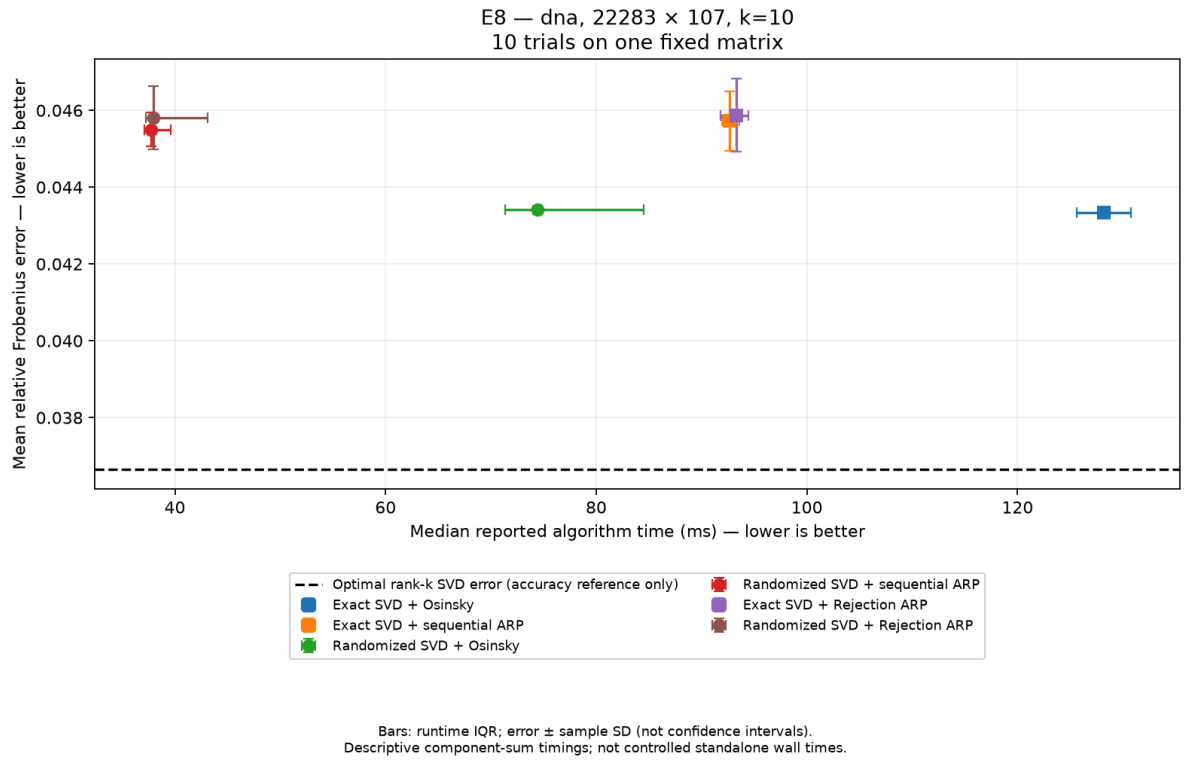


Fig. 4.14: Accuracy–runtime trade-off of the six CSSP pipelines on the DNA dataset.

The trade-off analysis shows that randomized-SVD pipelines achieve substantially lower runtimes than their deterministic-SVD counterparts. On the DNA dataset, Osinsky provides lower reconstruction error, while ARP and Rejection ARP provide faster execution at the cost of slightly higher error. Overall, the results demonstrate the main trade-off observed throughout the experiments: randomization can reduce computational cost, but the resulting accuracy depends on where randomization is introduced and on the dataset being evaluated.

5. Conclusion

This project investigated the effect of introducing randomization at different stages of a column subset selection problem pipeline for low-rank matrix approximation. Six pipelines were implemented by combining deterministic and randomized SVD with Osinsky, ARP, and Rejection ARP column-selection methods. The implementations were validated through independent reference calculations, numerical robustness checks, randomized sampling tests, and an automated test suite of 576 passing cases.

The experimental results showed that the effect of randomization depends strongly on where it is introduced. Randomized SVD provided the clearest computational benefit, substantially reducing basis-construction time as matrix size increased while generally maintaining comparable approximation quality. In contrast, randomizing the column-selection stage introduced trial-to-trial variability. Osinsky achieved the exact CSSP optimum in the small exhaustive experiment and produced lower reconstruction errors than ARP and Rejection ARP in several experiments, particularly on the DNA dataset. ARP and Rejection ARP nevertheless produced valid approximations and could reduce selection cost, with differences between the two becoming more relevant for larger candidate-column problems.

The experiments also demonstrated that algorithm performance depends on the characteristics of the input matrix. Matrix structure had a considerable influence on reconstruction accuracy, while sparse representation was most useful at low matrix densities. On the real-world datasets, the relative behavior of the selection methods differed: Osinsky gave lower errors on the DNA data, whereas the six pipelines produced nearly identical errors on the evaluated Meszaros subset. Across both datasets, randomized basis construction reduced runtime.

Overall, the study demonstrates that randomization is not simply an alternative that is uniformly better or worse than deterministic computation. Instead, it introduces a trade-off between computational efficiency, approximation quality, and variability. For the experiments conducted in this project, randomizing the SVD stage provided the most consistent runtime improvement, while randomizing column selection offered different computational trade-offs depending on matrix size and structure. These findings highlight the value of considering both the location of randomization within the CSSP pipeline and the properties of the input matrix when selecting an approach for practical low-rank approximation.

6. References

[1] A. Cortinovis and D. Kressner, “Adaptive randomized pivoting for column subset selection, DEIM, and low-rank approximation,” *SIAM Journal on Matrix Analysis and Applications*, vol. 47, no. 1, pp. 25–47, 2026.

[2] A. Cortinovis and D. Kressner, “Low-rank approximation in the Frobenius norm by column and row subset selection,” *SIAM Journal on Matrix Analysis and Applications*, vol. 41, no. 4, pp. 1651–1673, 2020.

[3] N. Halko, P.-G. Martinsson, and J. A. Tropp, “Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions,” *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011.

[4] E. N. Epperly, “Adaptive randomized pivoting and volume sampling,” *SIAM Journal on Matrix Analysis and Applications*, accepted for publication, 2026, arXiv:2510.02513.